
Discussion 9

Note: Your TA will probably not cover all the problems on this worksheet. The discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, discussions, and homework.

This Week's Cool AI Demo/Video:

Extropic TSU: <https://www.youtube.com/watch?v=Y28JQzS6TlE>

1 Convolution Operation in CNNs

In this question, we will look at the mechanism of weight sharing in convolutions. Suppose that we have an input vector with $D = 9$ dimensions:

$$\mathbf{x} = [x_1 \quad x_2 \quad \dots \quad x_9]^T.$$

We compute a 1D convolution with the kernel filter that has $K = 3$ weights (parameters):

$$\mathbf{k} = [k_1 \quad k_2 \quad k_3]^T.$$

- (a) What's the output dimension if we apply filter $\mathbf{k}$ with no padding and stride of 1? What's the first element of the output? What's the last element?

- (b) What's the output dimension if we apply filter $\mathbf{k}$ with padding of size 1 and stride of 2? What's the first element of the output? What's the second element?

- (c) Recall that CNN filters have the property of weight sharing, meaning that different portions of an image can share the same weight to extract the same set of features. A convolution is actually a linear operation, which we can express as $\mathbf{x}' = \mathbf{K}\mathbf{x}$ for some weight matrix $\mathbf{K}$. Find $\mathbf{K}$ for the convolution applied in part (a). What is its dimension?

- (d) Consider now a 2-dimensional example with the following kernel filter and image:

$$\mathbf{K} = \frac{1}{4} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Using no padding and a stride of 1, compute the output, and describe the effect of this filter.

- (e) We want to know the general formula for computing the output dimension of a convolution operation. Suppose we have a square input image of dimension $W \times W$ and a $K \times K$ kernel filter. If we assume stride of 1 and no padding, what's the output dimension W' ? What if we applied stride of s and padding of size p , how would the dimension change?

2 L2 Regularization & Weight Decay

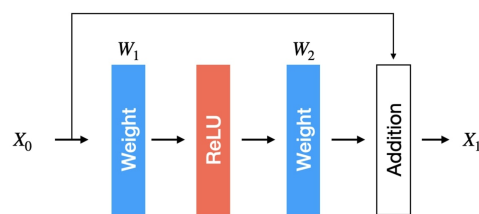
Derive the stochastic gradient descent (SGD) update rule when we add an L_2 regularization term $\lambda \|\mathbf{w}\|^2$ to a error function $E(\mathbf{w})$, and explain why we refer to this as weight decay regularization.

3 Vanishing Gradients & ResNets

The deeper a neural network grows, the more likely it will suffer from vanishing gradients because downstream layers (i.e., earlier layers) are the product of many backpropagated gradients—any of which may be close to zero. This question explores one approach to address vanishing gradients.

- (a) The vanishing gradient problem can arise from saturating activation functions, such as the sigmoid, $s(x) = \frac{1}{1+e^{-x}}$. What's the maximum gradient of the sigmoid function? Why might ReLU be more preferable than the sigmoid function in deep networks?

- (b) One common way to tackle vanishing gradients is by adding residual connections. The figure below shows a simplified example of a residual block. Assuming that the weights consist of an affine layer with zero biases, write an expression for X_1 in terms of W_1 , W_2 and X_0 .



- (c) Compute the gradient $\frac{\partial X_1}{\partial X_0}$. Based on what you see numerically, why does a residual connection preserve gradient norms better? (You may assume X_0 and X_1 are scalar.)

4 Dropout

In this question we explore dropout as a form of regularization. Recall that when we apply dropout during training, we randomly turn off individual nodes, as shown below.

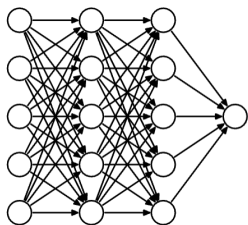


Figure 1: Original (and Final) Net

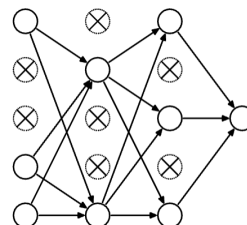


Figure 2: Net during Dropout Training

- (a) Explain qualitatively how dropout regularization works and why we can view dropout as simulating an ensemble of many “sub-networks.” Describe what happens during training when dropout is applied to a neural network’s neurons.
- (b) Dropout has a tendency to prevent any one weight from growing excessively large, compared to other weights in the network. Why would that be?
- (c) Consider a ReLU layer that computes $\mathbf{z} = h(\mathbf{W}\mathbf{x})$, where $\mathbf{x}$ represents the nodes in some layer, $\mathbf{z}$ represents the nodes in the subsequent layer, $\mathbf{W}$ is the weight matrix connecting those two layers, and $h(\cdot)$ is the ReLU activation function applied element-wise to a vector. Recall that the backpropagation step updates the weights in $\mathbf{W}$ by computing

$$\nabla_{\mathbf{w}_i} L = \frac{\partial L}{\partial z_i} h'(\mathbf{w}_i \cdot \mathbf{x}) \mathbf{x} \quad \text{and then} \quad \mathbf{w}_i \leftarrow \mathbf{w}_i - \eta \nabla_{\mathbf{w}_i} L,$$

where L is the loss function for a training point, $\mathbf{w}_i$ is the i th row of $\mathbf{W}$, and η is the learning rate/step size. If dropout turns off some fraction of the units in both layers of hidden units, how should we implement this backpropagation step?

- (d) Consider a single neuron receiving some input during training with dropout. Derive an expression for the expected output of this neuron (pre-activation) under dropout. (Assume a fixed input and weight configuration, and that each unit is dropped independently with a given probability. You may assume a “keep probability” p for each unit – i.e. each neuron is retained with probability p during training.) Show how this relates to the test-time behavior of scaling weights or activations.